

chapter4

제4장 Kafka 활용: 실시간 데이터 수집 및 토픽 설계

3장에서 파이프라인을 다섯 계층으로 나눌 때, 수집 계층의 대표 도구로 Kafka를 소개했다. 이 장은 그 수집 계층을 구현 수준으로 심화한다. 먼저 토픽·파티션·오프셋·컨슈머 그룹이라는 핵심 개념을 민원 이벤트 하나가 흘러가는 경로를 따라가며 이해하고, 공공 데이터 특성(기관·지역 단위 처리, 감사 추적, 버스트 유입)에 맞는 토픽 이름과 파티션 키를 설계한다. 이어서 생산자(Producer)와 소비자(Consumer)를 실제 브로커에서 실행해 기본 동작을 검증하고, 장애가 났을 때 커밋 시점에 따라 같은 시스템이 메시지를 잃기도 하고(유실) 두 번 처리하기도 하는(중복) 현상을 직접 관찰한다. 이 관찰이 다음 5장(중복 방지와 일관성 보장)의 출발점이 된다. 마지막으로 이 전달 보증(delivery guarantee)의 선택이 민간 실시간 현장에서 어떻게 다뤄지는지, 그리고 공공의 감사·책임성 요구가 그 선택을 어떻게 변형시키는지를 적용한다. 실습(★★★☆☆)은 3장에서 구성한 로컬 브로커를 재사용한다.

학습 목표

Kafka의 토픽, 파티션, 컨슈머 그룹을 자신의 말로 설명하고, 순서 보장이 어느 범위에서 성립하는지 판단할 수 있다.
공공 데이터 특성에 맞는 토픽 명명 규칙과 파티션 키 설계 원칙을 적용할 수 있다.
생산자(Producer)와 소비자(Consumer)의 기본 동작을 실행하고, 커밋 시점에 따라 유실·중복이 실제로 달라지는 것을 검증할 수 있다.
전달 보증 수준의 선택을 민간 실시간 패턴에서 배우고, 공공 감사·책임성 요구에 맞게 적용할 수 있다.
요구사항에서 토픽 설계표(이름·키·파티션 수·보존)를 도출하고, 각 값의 근거와 그 설계가 깨지는 조건을 적을 수 있다.

4.1 Kafka 핵심 개념

학습 목표

메시징 계층이 왜 필요한지 “속도 차이·버퍼·리플레이”로 설명한다.
토픽→파티션→오프셋→컨슈머 그룹의 관계를 이벤트 1건의 경로로 따라간다.

왜 메시징 계층이 필요한가

공공 실시간 데이터의 생산 속도와 소비 속도는 거의 항상 다르다. 재난 문자 발송 직후 민원이 집중되는 순간, 수집 API는 초당 수백 건을 받지만 하류의 처리 서버는 그 속도를 따라가지 못할 수 있다. 생산자(Producer, 예: 민원 접수 API — 이벤트를 만들어 로그에 써 넣는 쪽)와 소비자(Consumer, 예: 처리 서버·감사용 적재기 — 로그를 읽어 실제로 처리하는 쪽)를 직접 연결하면 이 속도 차이가 곧바로 장애(요청 거부, 데이터 유실)로 이어진다.

Kafka는 둘 사이에 **로그(log)** 를 둔다. 생산자는 이벤트를 로그에 추가하기만 하고, 소비자는 자기 속도로 로그를 읽는다. 이 단순한 구조가 세 가지 능력을 만든다.

버퍼: 소비가 지연되어도 이벤트는 로그에 안전하게 축적된다. 버스트를 흡수한다.

리플레이: 읽어도 지워지지 않으므로, 읽는 위치를 되돌려 과거 이벤트를 다시 처리할 수 있다. 감사 재현·재처리(3장 3.1)의 기술적 토대다.

분리(decoupling): 생산자는 소비자가 몇 개인지, 지금 살아 있는지 알 필요가 없다. 새 소비자(예: 감사용 적재기)를 생산자 수정 없이 추가한다.

이 “추가 전용 로그(append-only log)”라는 선택은 Kafka만의 독특함이 아니라 분산 데이터 시스템의 공통 토대다. Kleppmann은 *Designing Data-Intensive Applications*에서 복제(replication)와 스트림 처리의 근간을 “순서가 매겨진 불변 로그를 여러 소비자가 각자의 속도로 따라 읽는 것”으로 설명하는데, Kafka의 파티션이 정확히 그 로그이다(분산 시스템 일반 원칙을 차용한 배경 자료). 이 관점이 4.1의 뒤 내용을 예고한다 — 순서는 로그 하나(=파티션) 안에서만 정의되고, “어디까지 안전하게 읽었는가”의 기록(커밋)과 “응답을 받지 못했다”의 모호성(4.5)이 모두 이 로그 모델에서 파생된다.

핵심 개념: 이벤트 1건의 경로

“강남구에서 불법주차 민원 1건 접수”라는 이벤트가 Kafka를 통과하는 경로는 다음과 같다. 아래 용어는 이 장 전체에서 반복 사용된다.

토픽(topic): 이벤트를 종류별로 모아 담는, 이름이 붙은 단위다. 종류가 다른 이벤트(민원 접수, 재난 알림 등)는 서로 다른 토픽으로 나눠 보낸다. 실습에서는

`public.complaint.received.v1` 이라는 토픽 하나에 민원 접수 이벤트만 보낸다. 토픽 이름은 “이 토픽에는 무엇이 오는가”를 약속하는 계약(contract)이다.

파티션(partition): 토픽은 물리적으로 여러 파티션으로 나뉜다. 각 파티션은 **추가 전용(append-only)** 순서 로그다. 실습 토픽은 파티션 3개로 만든다.

키(key)와 파티셔닝(partitioning): 이벤트에 키를 부여하면, 같은 키는 항상 같은 파티션에 배정된다(기본 동작: 키 해시). 실습에서 키는 지역구다. 실제 실행에서 “강남구” 이벤트는 매번 파티션 1에, “관악구” 이벤트는 매번 파티션 0에 배정되었다.

오프셋(offset): 파티션 안에서 각 이벤트가 받는 순번이다. 오프셋은 파티션 안에서만 증가하므로, **순서 보장도 파티션 안에서만 성립한다**. “강남구 민원끼리는 접수 순서가 보존되지만, 강남구와 관악구 사이의 순서는 보장되지 않는다”가 정확한 문장이다.

브로커(broker): 파티션을 저장·서빙하는 서버 프로세스다. 운영 환경에서는 여러 브로커가 파티션을 복제(replication)해 장애를 견딘다. 실습은 브로커 1대(복제 계수 1)다.

컨슈머 그룹(consumer group): 같은 `group.id` 를 가진 소비자들은 파티션을 나눠 맡는다. 파티션 3개에 컨슈머 3대면 하나씩 맡아 병렬 처리한다. 그룹이 다르면 같은 토픽을 독립적으로 처음부터 읽을 수 있다 — 처리용 그룹과 감사 적재용 그룹이 서로를 방해하지 않는 이유다.

커밋된 오프셋(committed offset): 그룹은 “어디까지 읽었는지”를 브로커에 기록한다. 소비자가 중단되었다 재시작하면 이 기록부터 다시 읽는다. **언제 기록하느냐(커밋 시점)** 가 4.5절과 5장의 핵심 문제다.

보존 기간(retention): 이벤트는 소비 여부와 무관하게 설정된 기간 동안 보존된다. 보존 기간이 곧 리플레이 가능 범위다.

워크드 예제(worked example): 이벤트 4건을 실제로 따라가기

실습 4.1의 실제 실행 기록(`ch4_processed_g-baseline.jsonl`)에서 처음 4건의 경로를 그대로 옮기면 다음과 같다. 키는 지역구, 토픽은 파티션 3개다.

이벤트	키(지역구)	배정 파티션	받은 오프셋
#1 불법주차	중구	1	0
#2 도로파손	강남구	1	1
#3 도로파손	성동구	0	0
#4 가로등고장	중구	1	2

세 가지가 확인된다. 첫째, 오프셋은 파티션마다 따로 계수한다 — #3은 전체로는 세 번째 이벤트지만 파티션 0에서는 첫 번째(오프셋 0)다. 둘째, 같은 키는 같은 파티션에 배정된다 — 중구(#1, #4)는 둘 다 파티션 1이고, 그 안에서 순서(0→2)가 보존된다. 셋째, 중구와 강남구가 같은 파티션 1을 공유한다 — 키가 다르다고 파티션이 다르리라는 보장은 없다(키 해시가 우연히 같은 파티션을 가리킬 수 있다). 반대로 중구와 성동구 사이의 순서는 파티션이 달라 애초에 비교 대상이 아니다.

표 4.1 Kafka 핵심 개념 요약

개념	정의	운영 관찰 포인트
토픽	이벤트의 논리적 채널	명명 규칙, 스키마 계약
파티션	토픽을 나눈 순서 로그	개수(병렬성 상한), 부하 쏠림
키	파티션 배정 기준	같은 키→같은 파티션, 순서 보장 단위
오프셋	파티션 내 순번	커밋 위치, 컨슈머 랙(lag)
브로커	저장·서빙 서버	복제 계수, 디스크 사용량
컨슈머 그룹	파티션을 나눠 맡는 소비자 집합	리밸런싱(rebalancing), 그룹별 진행 위치
보존 기간	이벤트 보존·리플레이 범위	감사 요구와의 정합

공공 사례

지자체 민원 시스템에서 “실시간 현황 대시보드”와 “감사용 원본 적재”는 요구가 다르다. 대시보드는 최신 이벤트를 빠르게, 감사 적재는 모든 이벤트를 빠짐없이 읽어야 한다. 같은 토픽에 컨슈머 그룹을 둘 두면 두 요구를 한 수집 계층에서 처리할 수 있고, 감사에서 “특정 시점 데이터를 재현하라”는 요구가 오면 보존 기간 내에서 오프셋을 되돌려 리플레이한다.

핵심 정리

메시징 계층은 속도 차이를 흡수하는 버퍼이자, 리플레이를 가능하게 하는 로그다.
순서 보장의 단위는 토픽이 아니라 파티션이고, 파티션 배정은 키가 결정한다.
컨슈머 그룹은 병렬 처리(그룹 내 분담)와 독립 소비(그룹 간 격리)를 동시에 제공한다.

4.2 공공 데이터 토픽 명명과 파티션 키 설계

학습 목표

토픽 이름을 “나중에 검색·감사·승인이 가능한” 규칙으로 짓는다.
파티션 키 선택이 곧 순서 보장 단위의 선택임을 이해하고 트레이드오프를 판단한다.

토픽 명명: 이름이 곧 거버넌스다

토픽은 한 번 만들면 여러 시스템이 의존하는 계약이 된다. 이름에 규칙이 없으면 “이 토픽에 개인정보가 있는가”, “누가 주인인가”, “스키마가 바뀌면 누구에게 알리는가” 같은 질문에 답할 수 없게 된다. Kafka가 명명 표준을 강제하지는 않으므로, 조직이 규칙을 정해야 한다. 이 교재는 다음 예시 규칙을 사용한다(표준이 아니라 합의의 예다).

{도메인}.{데이터셋}.{이벤트}.{버전}
예: public.complaint.received.v1

도메인/데이터셋: 소유 조직과 데이터 종류를 드러낸다. 접근 권한·개인정보 등급을 토픽 단위로 묶는 기준이 된다.

이벤트: received, updated 처럼 “무슨 일이 있었는가”를 과거형으로 적으면 토픽이 상태 저장소가 아니라 사건 기록임이 분명해진다.

버전: 스키마가 호환 불가능하게 바뀔 때 v2 토픽을 새로 만들어 신·구 소비자를 공존시킨다. 스키마 진화의 체계적 관리는 보충(Schema Registry)에서 다룬다.

같은 규칙이 민간에서도 그대로 사용된다. 이커머스라면 도메인 자리에 사업 단위가 들어가 commerce.order.created.v1, commerce.payment.completed.v1 이 된다 — 구조는 같고 도메인 어휘만 바뀐다. 다른 것은 규칙을 정하는 주체다. 공공에서는 개인정보 등급·소관 부서가 이름에 반영되어야 하므로 규칙이 기관 표준으로 정해지고, 민간에서는 팀 간 협상과 스키마 레지스트리로 정한다.

이 명명 규칙은 1장에서 말한 **데이터 계약(data contract)** — 생산자와 소비자가 필드·의미·품질·변경 절차를 합의해 문서와 코드로 관리하는 약속 — 의 Kafka판이다(1장 1.1.5 참조). 토픽 이름은 그 계약의 표지이고, 스키마(버전)는 계약의 본문이다. 계약이 없는 토픽은 이름만 있는 빈 약속이라, 소비자가 필드 하나의 의미를 오해해도 시스템은 중단되지 않고 조용히 틀린 결과를 산출한다 — 1장에서 침묵 실패(silent failure)라 부른 그 실패가 수집 계층에서 발생하는 지점이다.

파티션 키: 순서 보장 단위의 선택

4.1에서 순서는 파티션 안에서만 보장된다고 했다. 따라서 파티션 키 선택은 기술 설정이 아니라 “무엇끼리 순서가 지켜져야 하는가”라는 업무 요구의 전환이다.

민원 이벤트의 키를 지역구로 하면: 지역구별 접수·처리 순서가 보존된다. 지역구 단위 집계·감사에 유리하다.

키를 민원 건 ID로 하면: 같은 민원의 상태 변화(접수→배정→처리) 순서가 보존된다. 건 단위 이력 추적에 유리하다.

키를 지정하지 않으면: 이벤트가 파티션에 분산되어 부하는 고르지만, 어떤 순서도 보장되지 않는다.

키 선택에는 대가가 있다. 키 분포가 치우치면 특정 파티션에 부하가 몰린다(**핫 파티션**). 구체적인 시나리오는 다음과 같다 — 평시에는 지역구 키에 따라 8개 구가 3개 파티션에 고르게 분산되지만, 특정 지역구에 재난이 발생해 그 구의 민원이 초당 수백 건으로 폭증하면, 지역구 키를 사용하는 한 그 이벤트가 **모두 한 파티션에** 배정된다. 같은 키는 같은 파티션이라는 성질 때문에, 버스트 상황에서는 한 파티션만 과부하되고 나머지 두 파티션의 자원은 활용되지 않는다. 그 파티션을 맡은 컨슈머 하나가 지연되면 그 지역구의 민원 처리 전체가 지연된다. 재난 대응처럼 시급성이 높은 시점일수록 이 지연의 영향이 크다. 순서 보장 범위를 좁힐수록(더 세밀한 키, 예: 민원 건 ID) 부하는 고르게 퍼지지만, 넓은 범위의 순서(지역구별 접수 순서)는 포기하게 된다. 어느 쪽이 옳은가는 기술이 아니라 “재난 시 무엇을 지켜야 하는가”라는 업무 판단이다.

키 선택의 구조는 도메인을 가리지 않는다. 이커머스에서 파티션 키를 주문 ID로 잡으면 같은 주문의 상태 변화(생성→결제→배송) 순서가 보존되고, 고객 ID로 잡으면 같은 고객의 주문들이 순서대로 처리된다 — 민원 건 ID와 지역구의 선택과 같은 구조다. 핫 파티션도 함께 수반된다. 재난 시 한 구의 민원이 폭증하는 것과, 플래시 세일 시작 시각에 특정 상품 ID로 주문이 물리는 것은 같은 현상이다. 다만 대가의 종류가 다르다 — 민원 지연은 시민의 사건 처리가 늦어지는 것이고, 주문 지연은 매출과 이탈이다.

또 하나의 주의점은 파티션 수 변경이다. 파티션을 늘리면 키 해시→파티션 매핑이 바뀌어, 같은 키의 옛 이벤트와 새 이벤트가 다른 파티션에 놓인다. 키 기반 순서에 의존하는 시스템이라면 초기 설계에서 파티션 수에 여유를 두는 이유다.

토픽 설계표

아래는 이 교재의 공공 데이터 예시에 대한 토픽 설계표다. 파티션 수와 보존 기간은 검증된 성능 수치가 아니라 **요구사항에서 도출한 예시 설계값**이며, 실제 값은 부하 테스트(12장)와 감사 요구로 확정한다.

표 4.2 공공 데이터 토픽 설계표(예시)

토픽 이름	파티션 키	순서 보장 단위	파티션 수 (예시)	보존(예시)	설계 근거
<code>public.complaint.received.v1</code>	지역구	지역구별 접수 순서	3	7일	지역 단위 집계·감사, 재처리 여유
<code>public.air.measured.v1</code>	측정소 ID	측정소별 시계열 순서	6	30일	측정소 단위 드리프트 분석(2장), 월간 보고 재계산
<code>public.disaster.alerted.v1</code>	재난 유형	유형별 발령 순서	3	90일	사후 감사·재현 요구가 길다

보존 기간을 정하는 방식도 도메인에 따라 달라진다. 공공은 법적 보관 의무나 감사 주기가 **하한**을 정하고, 그 아래로는 내려갈 수 없다. 민간은 하한을 정해 주는 조문이 없어 스토리지 비용과 리플레이 필요 기간의 균형으로 정한다. 그래서 같은 토픽 설계표라도 공공에서는 “보존 7일”의 근거 칸에 감사 주기가 들어가고, 민간에서는 “며칠분을 다시 처리할 수 있어야 하는가”가 들어간다.

검수 포인트(이 절에서 도출되는 질문)

토픽 명명 규칙이 문서로 존재하고, 실제 토픽 목록이 그 규칙을 따르는가.
 각 토픽의 파티션 키 선택에 “순서 보장 단위” 근거가 적혀 있는가 — “기본값이라서”는 근거가 아니다.

보존 기간이 감사·재처리 요구에서 역산되었는가, 디스크 사정으로 정해졌는가.

공공 사례

공공 데이터에는 “기관·지역·측정소” 같은 자연스러운 처리 단위가 이미 있고, 감사 요구가 보존 기간을 결정하는 경우가 많다. 예컨대 재난 대응 기록은 사후 조사 기간을 견딜 만큼 보존해야 하므로, 보존 기간은 스토리지 비용이 아니라 감사 요구(3장 3.3)에서 역산한다. 명명 규칙·키·보존을 담은 토픽 설계표는 그 자체가 발주·검수 문서의 일부가 된다(부록 D).

핵심 정리

토픽 이름은 소유·내용·버전을 드러내는 거버넌스 장치이자 데이터 계약의 표지다. 규칙은 조직의 합의로 정한다.

파티션 키는 순서 보장 단위의 선택이며, 순서 범위와 부하 분산은 맞바꿈 관계다 — 재난 버스트 상황에서 이 상충 관계가 명확해진다.

파티션 수·보존 기간은 성능·감사 요구에서 도출하고, 설계표로 문서화한다.

표 4.2는 완성된 예시다. 같은 형식의 설계표를 직접 채우는 것이 실습 4.1의 1단계이며, 거기서 선택한 값이 실행 결과와 어떻게 대응하는지는 3단계에서 확인한다.

4.3 Producer 기본 구현

학습 목표

Producer의 전송 경로(직렬화→파티셔닝→전송→확인)를 이해한다.
확인 수준(acks) 설정이 유실 위험과 어떻게 연결되는지 판단한다.

핵심 개념

Producer가 이벤트 1건을 보낼 때 내부에서 수행되는 처리는 네 단계다.

직렬화(serialization): 파이썬 딕셔너리 같은 객체를 바이트로 바꾼다. 실습은 JSON을 UTF-8로 인코딩한다.

파티셔닝: 이벤트에 키가 있으면, 그 키 값을 해시 함수(hash function, 입력값을 정해진 범위의 숫자로 바꾸는 계산)에 넣어 숫자로 바꾸고, 그 숫자를 파티션 개수로 나눈 나머지로 파티션 번호를 정한다. 같은 키는 항상 같은 나머지가 나오므로 항상 같은 파티션에 배정된다 (4.1의 “강남구→파티션 1” 예시가 이 계산 결과다). 이 계산은 Producer 쪽에서 일어난다 — 브로커가 정해 주는 것이 아니다.

전송: 레코드를 브로커에 보낸다. 실제로는 배치로 묶어 보내며, 전송은 비동기다.

확인(acknowledgement): 브로커가 기록을 확인해 주는 수준을 `acks` 로 정한다. 이 설정이 “전송 성공”의 의미를 결정한다.

표 4.3 acks 설정 비교

설정	의미	유실 위험	용도
<code>acks=0</code>	확인을 기다리지 않음	브로커 장애 시 유실	손실 허용 메트릭 등
<code>acks=1</code>	리더(leader) 파티션 기록만 확인	리더 교체 직후 유실 가능	지연·안전의 절충
<code>acks=all</code>	동기화된 복제본 전체 확인	가장 낮음	민원·재난 등 유실 불가 데이터

공공 데이터에서 “놓치면 안 되는” 이벤트(민원, 재난)는 `acks=all` 이 출발점이다. 다만 acks는 브로커까지의 유실만 다루며, 재시도로 인한 중복이나 소비 단계 유실은 별개 문제다(4.5절).

먹등 Producer: acks가 방지하지 못하는 중복을 막는다

`acks=all`은 “브로커에 확실히 기록되었는가”만 보장한다. 그런데 브로커가 기록을 마친 뒤 **확인 응답만 네트워크에서 유실**되면, Producer는 실패로 판단하고 같은 메시지를 다시 보낸다 — 브로커에는 같은 이벤트가 두 번 기록된다. 안전한 설정(`acks=all`)일수록 재시도가 적극적이라 이 중복은 오히려 더 자주 발생한다. 여기에 대한 브로커 수준의 해법이 **먹등 Producer**(`enable.idempotence=true`)다. Producer마다 고유 ID와 시퀀스 번호를 부여해, 브로커가 “이미 받은 시퀀스의 재전송”을 식별해 폐기한다. 최근 Kafka는 이 보호를 기본값으로 상향했다 — KIP-679는 Producer 기본값을 “가장 강한 전달 보증”으로 바꿔 `enable.idempotence=true` 와 `acks=all` 을 기본으로 켜다(Kafka 공식 KIP — 직접 선례). 즉 오늘날의 Producer는 별도 설정 없이도 재시도 중복을 브로커 안에서 제거한다. 다만 이 먹등성은 **Producer 세션 하나의 재시도만** 막는다 — 애플리케이션이 재시작해 처음부터 다시 보내거나, 소비 측 커밋 시점 때문에 생기는 중복은 이 장치의 밖이다(4.5.5장). 순서 보장 조건(`max.in.flight.requests`)과 exactly-once로의 확장은 보충에서 다룬다.

구현과 실행

실습 Producer의 핵심은 세 줄이다. 지역구를 키로 지정하고, `acks="all"` 로 전송하고, 확인(ack)을 기다린다.

```
producer = KafkaProducer(bootstrap_servers=..., acks="all", ...)
future = producer.send(topic, key=event["district"], value=event)
meta = future.get(timeout=15) # 브로커 확인 대기 → 파티션·오프셋 회신
```

전체 코드는 `practice/chapter4/code/4-1-kafka-producer.py` 참고

실행 결과(실제)

민원 이벤트 200건을 `public.complaint.received.v1` (파티션 3개)에 전송했다(출력: `practice/chapter4/data/output/ch4_produce_baseline.json`).

전송 200건, 소요 0.301초(로컬 브로커, 건별 확인 대기 포함)
파티션 분포: 파티션 0에 61건, 파티션 1에 76건, 파티션 2에 63건
지역구→파티션 매핑: 강남구·마포구·중구→1, 관악구·성동구→0, 송파구·용산구·종로구→2 —
8개 지역구 전부 단일 파티션에만 배정(`key_to_single_partition: true`)

관찰

“같은 키→같은 파티션”이 실제로 성립한다. 지역구별 순서 보장의 물리적 근거다.
파티션 분포(61/76/63)는 고르지 않다. 키 해시는 지역구를 파티션에 나눌 뿐, 지역구별 이벤트 양까지 균등하게 만들지는 않는다 — 핫 파티션 논의(4.2)가 실측으로 확인된다. ###
핵심 정리
파티션 배정은 Producer의 키 해시가 결정하며, 실행 결과로 검증할 수 있다.
acks는 “전송 성공”의 정의다. 유실 불가 데이터는 `acks=all` 에서 출발한다.
acks는 브로커까지의 유실만, 멱등 Producer는 세션 내 재시도 중복만 책임진다. 앱 재시작.
소비 유실은 다른 층의 문제다.

4.4 Consumer 기본 구현

학습 목표

컨슈머 그룹 참여→파티션 할당→폴링→커밋의 순환을 이해한다.
자동 커밋과 수동 커밋의 차이가 왜 중요한지 설명한다.

핵심 개념

Consumer의 동작은 네 단계의 순환이다.

그룹 참여(join): `group.id` 를 갖고 그룹에 들어간다. 브로커의 코디네이터가 그룹 구성원에게 파티션을 나눠 준다(**할당**).

폴링(poll): 할당받은 파티션에서 레코드 배치를 가져온다. Kafka는 브로커가 전송하는(push) 방식이 아니라 소비자가 가져오는(pull) 방식이다 — 소비 속도를 소비자가 결정하는 구조다.

처리: 가져온 레코드로 업무 로직(저장, 집계, 알림)을 수행한다.

커밋(commit): “여기까지 읽었다”를 브로커에 기록한다. 재시작 시 이 위치부터 읽는다.

커밋에는 두 방식이 있다. **자동 커밋**(`enable.auto.commit=true`)은 일정 주기로 백그라운드에서 커밋한다 — 간편하지만 “처리 완료”와 무관한 시점에 커밋되므로, 장애 시 유실도 중복도 발생할 수 있다. **수동 커밋**은 코드가 커밋 시점을 정한다. 실습은 수동 커밋을 사용하며, 4.5절에서 커밋 시점을 바꿔 가며 결과 차이를 관찰한다.

소비가 생산을 따라가고 있는지는 **컨슈머 랙(consumer lag)** 으로 관찰한다. 랙은 "파티션의 마지막 오프셋 - 그룹이 커밋한 오프셋", 즉 아직 처리하지 못하고 쌓여 있는 이벤트 수다. 랙이 0 근처에서 유지되면 소비가 생산을 따라가는 것이고, 랙이 계속 증가하면 소비자가 지연되고 있다는 뜻이다. 랙이 중요한 이유는 **장애가 발생하기 전에 먼저 나타나는 신호**이기 때문이다 — 처리량이 생산량에 조금씩 못 미치기 시작하면 랙은 오류 하나 없이 서서히 늘고, 임계에 도달하기 훨씬 전부터 추세로 드러난다. 그래서 랙은 운영에서 가장 먼저 대시보드에 올리는 조기 경고 지표(SLI)이고, 11장 모니터링에서 재학습·증설 판단의 핵심 지표로 다시 등장한다.

랙이 증가할 때 실제로 지연되는 것이 무엇인지는 도메인에 따라 달라진다. 민원 파이프라인에서 랙 증가는 접수 확인과 담당 배정이 지연되는 것이고, 이커머스에서는 주문 확인 알림이 지연되는 것이며, 재난 경고에서는 경고 발령 자체가 지연되는 것이다. 같은 지표가 어떤 곳에서는 고객 불만이고 어떤 곳에서는 인명 위험이라, 랙의 경고 임계값은 기술이 아니라 업무 요구에서 나온다.

4.1의 공공 사례에서 대시보드용과 감사 적재용으로 그룹을 나눈 것도 랙과 함께 봐야 한다. 민간에서는 같은 자리에 데이터 레이크·웨어하우스 적재 그룹이 들어가고, 그 원본이 BI와 ML 학습 데이터의 출발점이 된다. 목적은 감사와 분석으로 갈리지만 구조는 같다 — 업무 처리의 사정(재시도, 필터링, 스키마 변환)에 오염되지 않은 원본을 따로 확보하는 것이다. 그룹이 나뉘면 랙도 나뉜다. 처리용 그룹이 지연되거나 중단되어도 적재용 그룹의 랙은 별개로 변동하므로, 둘을 따로 감시하지 않으면 한쪽의 정체를 놓친다.

컨슈머가 응답 없이 중단되면 어떻게 되는가. 컨슈머는 브로커에 주기적으로 **하트비트(heartbeat)** 를 보내고, **세션 타임아웃(session timeout, 실습 설정 10초)** 동안 하트비트가 없으면 브로커가 그 컨슈머를 그룹에서 제외하고 파티션을 다른 구성원에게 재할당한다(**리밸런싱**). 이 구조의 함의는 실습에서 직접 확인한다: 크래시(crash) 직후 재시작한 컨슈머는 중단된 이전 인스턴스가 세션 타임아웃으로 제외될 때까지 파티션을 받지 못할 수 있다.

구현과 실행

실습 Consumer의 핵심은 "수동 커밋 + 폴링 루프"다.

```
consumer = KafkaConsumer(topic, group_id=..., enable_auto_commit=False,
                           auto_offset_reset="earliest", ...)
batch = consumer.poll(timeout_ms=1000) # 배치 당겨오기
consumer.commit()                     # 커밋 시점은 코드가 결정
```

전체 코드는 [practice/chapter4/code/4-2-kafka-consumer.py](#) 참고

실행 결과(실제)

Producer(4.3)와 동시에 실행한 Consumer가 200건을 소비했다(출력:

`practice/chapter4/data/output/ch4_consume_g-baseline.json`).

수신·처리 200건 — 유실 0, 중복 0

종단 지연(생산 시각→소비 시각): 평균 3.5ms, 최대 110.7ms

관찰

중단 지연 평균 3.5ms는 **컨슈머가 먼저 실행되어 대기하고 있을 때**의 값이다. 이 실험은 컨슈머를 먼저 시작하고 8초 뒤에 생산을 시작했다. 반대로 컨슈머가 생산 이후에 시작하면, 지연은 브로커 성능이 아니라 “소비 시작이 얼마나 늦었는가”가 지배한다 — 2장의 이벤트 시간과 처리 시간 구분이 그대로 재등장한다. 운영에서 컨슈머 락(lag) 모니터링(11장)이 필요한 이유다.

최대 110.7ms가 평균(3.5ms)의 약 32배다. 처리 기록을 확인하면 이 최댓값은 **가장 첫 레코드**에서 나왔다 — 첫 폴링에는 파티션 할당·패치 준비 같은 시동 비용이 추가된다. 지연은 평균만으로 판단하지 않으며(12장에서 분위수로 확장), 최댓값이 “언제” 나왔는지가 원인 진단의 단서가 된다. ### 핵심 정리

Consumer는 할당→폴링→처리→커밋의 순환이며, 커밋 위치가 재시작 시의 출발점이다.

자동 커밋은 커밋 시점을 통제할 수 없어, 유실·중복이 모두 가능해진다.

컨슈머 락은 오류보다 먼저 나타나는 조기 경보 지표이며, 세션 타임아웃·리밸런싱은 재시작 지연의 원인이다.

4.5 실패와 재시도 흐름

학습 목표

유실·중복·지연이 파이프라인의 어느 단계에서 왜 발생하는지 파악한다.

커밋 시점 선택이 at-most-once/at-least-once를 결정함을 이해한다.

핵심 개념: 실패는 어디서 생기는가

“메시지가 없어졌다/두 번 왔다”는 증상은 하나지만 원인 지점은 여러 곳이다. 단계별로 나누면 문제가 선명해진다.

표 4.4 단계별 유실·중복·지연 발생 지점

단계	유실	중복	지연
Producer→브로커	acks=0/1 에서 브로커 장애	확인 응답 유실 후 재시도 (브로커엔 이미 기록됨)	배치 대기, 브로커 과부하
브로커	복제 부족 상태의 디스크 장애	—	—
브로커→Consumer	커밋 먼저, 처리 전 크래시	처리 먼저, 커밋 전 크래시	컨슈머 락, 리밸런싱 정지
Consumer→저장소	저장 실패를 무시하고 커밋	저장 성공 후 커밋 실패	저장소 병목

주목할 것은 중복의 두 원천이다. Producer 쪽 중복은 재시도라는 **안전장치의 부작용**이다 — 확인 응답이 유실되면 Producer는 이미 기록된 메시지를 다시 보낸다(먹등 Producer가 이를 브로커 수준에서 제거한다 — 4.3). Consumer 쪽 중복은 **커밋 시점**의 문제다. 그리고 커밋 시점은 코드로 선택하는 것이다.

재시도는 왜 불가피하게 중복을 발생시키는가

표 4.4의 “확인 응답 유실 후 재시도”를 한 겹 더 분석하면, 이것이 Kafka의 결함이 아니라 분산 시스템의 근본 한계임이 드러난다. Producer가 메시지를 보내고 응답을 기다리는데 시간이 지나도 응답이 없다. 이때 Producer가 아는 것은 오직 “응답이 오지 않았다”뿐이다 — 브로커가 **기록에 실패한 것인지**(그렇다면 재전송해야 한다), 아니면 **기록은 됐는데 응답만 유실된 것인지**(그렇다면 재전송은 중복이다) 구분할 방법이 없다. Kleppmann은 *Designing Data-Intensive Applications*에서 이 모호성을 분산 시스템의 일반 성질로 다룬다 — 네트워크 너머의 침묵은 “실패”와 “성공했으나 응답 유실”을 구분해 주지 않는다(배경 자료: 분산 시스템 일반 원칙). 재전송을 포기하면 유실을 감수하는 것이고, 재전송하면 중복을 감수하는 것이다. 그래서 유실이 더 비싼 시스템(민원·재난)은 필연적으로 중복을 택하게 되며, 이 “구분 불가능성”이 5장에서 다룰 중복 제거의 근본 이유가 된다.

커밋 시점이 전달 의미론을 결정한다

수동 커밋 컨슈머가 배치를 받았다고 가정한다. 커밋과 처리, 두 동작의 순서는 둘 중 하나다.

커밋 먼저, 처리 나중(at-most-once): 처리 중 크래시하면, 재시작한 컨슈머는 커밋 이후부터 읽으므로 처리 못 한 레코드는 다시 오지 않는다. **최대 한 번 — 유실 가능, 중복 없음.**

처리 먼저, 커밋 나중(at-least-once): 커밋 전 크래시하면, 재시작한 컨슈머는 옛 커밋 위치부터 다시 읽으므로 이미 처리한 레코드가 다시 온다. **최소 한 번 — 유실 없음, 중복 가능.**

유실과 중복을 실제로 얼마나 겪고 있는지는 지표로 봐야 한다. 중복은 중복 제거 키의 충돌 건수로 계수하면 되지만(5장), **유실은 계수하기 어렵다** — 실습 시나리오 B에서 확인되듯 오류 로그가 남지 않기 때문이다. 그래서 유실은 생산 건수와 처리 건수를 맞춰 보는 **대사(reconciliation)** 로만 잡힌다. 시간 창을 정해 “이 창에서 Producer가 보낸 건수”와 “소비자가 처리한 고유 건수”를 비교하고 차이가 0이 아니면 조사하는 방식이다. 이 대사를 자동화하지 않으면 유실은 감사 시점이나 민원 항의로 발견된다 — 민간에서는 정산 불일치나 광고주 클레임이 같은 자리를 차지한다.

커밋 시점의 선택은 그래서 기술 취향이 아니라 업무 판단이다. 장바구니 이탈 쿠폰 알림을 예로 들면, 알림을 보낸 뒤 커밋하면(at-least-once) 크래시 시 같은 사용자에게 쿠폰이 두 번 갈 수 있고, 커밋 후 보내면(at-most-once) 크래시 시 쿠폰이 아예 안 갈 수 있다. 쿠폰 하나 더 발송되는 손실과 고객 한 명을 놓치는 손실을 비교해 대개 전자를 택한다. 재난 문자에서는 같은 비교의 답이 다르다 — 중복 경보의 신뢰 훼손도 실질 비용이라, 어느 쪽도 감내하지 않고 발송 게이트웨이에 먹등 키를 부여해 양쪽을 모두 차단한다. 판단의 형식은 같고 답이 도메인마다 다르다.

“정확히 한 번”은 전달이 아니라 반영이다

그렇다면 “정확히 한 번(exactly-once)”은 왜 세 번째 선택지로 간단히 선택할 수 없는가. 여기에는 혼란의 오해가 있다. 네트워크 위에서 메시지가 물리적으로 “정확히 한 번 전달”되도록 보장하기는 근본적으로 어렵다 — 위의 ack 모호성이 사라지지 않기 때문이다. Kafka가 제공하는 exactly-once semantics는 “전달”이 아니라 **“여러 번 전달되어도 정확히 한 번 반영된 것과 같은 결과”**를 뜻한다. 이를 만드는 두 축이 있다. 하나는 맥등 Producer(4.3)로 Producer 재시도 중복을 브로커가 제거하는 것, 다른 하나는 **트랜잭션**으로 “읽고-처리하고-쓰고-오프셋 커밋”을 하나의 원자 단위로 묶어 부분 성공을 없애는 것이다. 이 트랜잭션 기능이 Kafka에 들어온 설계 문서가 KIP-98(Exactly Once Delivery and Transactional Messaging)이며(직접 선험), Confluent의 기술 블로그가 그 구현 원리를 상세히 해설한다(배경·구현 해설). 다만 exactly-once는 소비 종단(저장소)까지의 협조가 있어야 완성되고 처리량 비용도 따른다. 그래서 **실무 기본값은 at-least-once를 선택하고 중복을 하류에서 제거**하는 것이며, 그 하류 설계 — 중복 제거 키와 저장소 맥등 쓰기(UPSERT) — 가 5장 전체의 주제다. 5장은 4장이 남긴 실측 중복 스트림 전체를 다시 재생해도 업무 상태 지문 (eed1522a7ca844bd)이 변하지 않음을, 즉 at-least-once 위에서 “정확히 한 번 반영”이 성립함을 실측으로 보인다.

공공 사례

민원 처리 시스템에서 at-most-once를 사용하면 “접수했는데 처리 기록이 없는 민원”이 발생한다 — 공공 서비스에서 수용하기 어려운 실패다. at-least-once를 사용하면 같은 민원이 두 번 처리 목록에 오를 수 있다 — 불편하지만 중복 제거로 교정 가능한 실패다. 두 실패의 비대칭(유실은 복구 불능, 중복은 교정 가능)이 공공 데이터에서 at-least-once가 기본값인 이유다.

현장 포스트모템: 자동 커밋이 전달 보증을 조용히 강등시킨다

이 절의 이론이 실제로 어떻게 사고가 되는지는 이 책 뒤편에서 실측으로 확인된다. 14장의 통합 시스템 구축 중 발견된 사건이 대표적이다 — 소비자에 `enable_auto_commit=True`가 켜져 있으면, 자동 커밋이 내구 쓰기(저장소에 실제로 반영)가 끝나기 **전에** 오프셋을 밀어 올릴 수 있다. 그러면 크래시 시 그 메시지는 이미 “읽은 것”으로 표시되어 재생되지 않는다 — at-least-once를 의도했는데도 코드가 조용히 **at-most-once로 강등**되어 유실이 발생하는 것이다. 오류 로그 한 줄 없이 진행된다. 14장의 처리기는 이 유실 문제를 방지하려고 자동 커밋을 끄고, 내구 쓰기가 끝난 뒤에만 수동 커밋한다. “그러면 크래시 시 재생(중복)이 발생하지 않는가”가 다음 질문인데, 그 재생을 무해하게 만드는 것이 5장의 맥등 설계다. 실제로 14장의 중복 재전송 드릴에서 물리 수신 28건 중 흡수된 중복은 11건(수송 계층 재전송 10 + 원천 파일 안의 중복 접수 1)이었고, 그래도 최종 집계는 7장 확정치와 동일했다 — **전달 보증과 맥등은 별개 기능이 아니라 한 쌍**이라는 것을 확인하는 실측이다.

핵심 정리

유실·중복·지연은 단계별로 원인이 다르며, 표 4.4처럼 지점을 나눠 진단한다.

커밋과 처리의 순서가 at-most-once(유실 가능)/at-least-once(중복 가능)를 결정하고, 설정 한 줄(자동 커밋)이 그 의도를 조용히 뒤집을 수 있다.

exactly-once는 "정확히 한 번 전달"이 아니라 "정확히 한 번 반영"이며, 공공 데이터의 기본값은 at-least-once + 하류 멍등 제거다(5장).

4.6 같은 설계, 다른 책임: 민간 패턴과 공공 요구가 만나는 지점

학습 목표

민간 현장의 실시간 이벤트 처리 패턴을 공공 대응물로 적용하고, 그 역방향으로도 옮긴다. 전달 보증 수준의 선택이 공공의 감사·책임성 요구와 어떻게 결합하는지 판단한다.

실시간 이벤트 파이프라인은 광고·커머스·핀테크 같은 민간 영역의 기술로 여겨지곤 하지만, 전달 보증의 선택이라는 핵심 판단은 도메인을 가리지 않는다. 다른 점은 민간에서 그 선택이 **비용 대 정확도의 사업 판단**인 데 비해, 공공에서는 그 위에 **감사·책임성·형평성**이 추가된다는 것이다. 아래 표는 민간의 대표 패턴을 공공 대응물로 옮기고, 각 경우에 적합한 전달 보증과 공공 특수성이 요구하는 변형을 정리한다.

표 4.5 민간 실시간 패턴의 공공 적용

민간 현장 패턴	무엇을 하는가	공공 대응물	적합한 전달 보증	공공 특수성에 따른 변형
광고 노출·클릭 실시간 집계 (최적화용)	노출·클릭을 초당 집계해 성과 지표· 최적화에 사용	민원 유입 실시간 현황판	최적화용 근사 집계는 일부 유실 (at-most-once)도 감내하나, 민원은 at-least-once + 하류 dedup	집계 수치가 인력 배치·예산 근거가 됨 → 중복 제거의 증거(총계 보존식) 까지 산출물로(7장)
주문·결제 이벤트 파이프라인	주문 1건도 잃지 않고 정확히 한 번 반영	재난 문자·센서 이벤트 수집	상태 반영은 exactly-once(먹등 쓰기), 외부 발송은 별도 먹등 장치	유실은 인명 위험, 중복 경보는 신뢰 훼손 → 양방향 비대칭을 모두 관리, 감사 재현 위해 리플레이 보존을 길게(표 4.2의 90일)
실시간 추천 이벤트(사용자 행동)	라이브 행동으로 추천 피처를 즉시 갱신	방문 민원(대면 접수)·콜센터· 온라인 채널 통합 이벤트	at-least-once + 업무 식별자 먹등 키	같은 민원이 여러 채널로 중복 유입 → <code>complaint_id</code> 먹등 키로 흡수, 형평 위해 채널별 처리 격차를 지표로 감시

세 사례에서 같은 원리가 서로 다른 방식으로 적용된다. **광고 대 민원**은 “유실을 얼마나 감내하는가”가 도메인에 따라 달라진다는 것을 보여준다 — 광고 노출 집계는 0.1%가 누락되어도 사업 판단이 달라지지 않지만, 민원 1건의 유실은 곧 처리 누락이고 시민 한 명의 사건이다. 그래서 같은 “실시간 집계”라도 공공은 더 강한 보증에서 출발하고, 나아가 “이 수치는 중복이 제거된 값인가”라는 감사 질문에 답할 증거(총계 보존식, `seen_count`)까지 산출물로 요구한다.

주문 대 재난은 비대칭이 양방향으로 걸리는 드문 경우다. 커머스에서는 보통 유실이 중복보다 비싸다(주문을 잃으면 매출과 신뢰를 잃는다). 재난 경보는 유실도 중복도 모두 비싸다 — 경보를 놓치면 인명 위험이고, 같은 경보가 여러 번 발송되면 반복된 오경보로 시스템 신뢰가 저하된다. 그래서 재난 이벤트는 `exactly-once` 반영을 지향하되, 사후 감사가 “그때 어떤 경보가 언제 나갔는가”를 재현할 수 있도록 리플레이 보존 기간을 길게 잡는다. 한 가지 경계할 점은, Kafka의 `exactly-once`는 브로커·저장소의 **상태 반영**까지를 원자적으로 묶을 뿐, 실제 재난 문자 발송 같은 **외부 부작용**의 중복 방지까지 자동으로 보장하지는 않는다는 것이다 — 문자 발송이 성공한 뒤 오프셋 커밋 직전에 크래시하면 재시작 후 같은 문자가 다시 발송될 수 있다. 외부 발송의 중복은 발송 명령에 먹등 키를 부여하거나(같은 경보 ID의 재발송을 게이트웨이(gateway)가 무시), 아웃박스(outbox) 패턴으로 “발송 여부”를 상태로 남겨 별도로 막아야 한다.

추천 대 채널 통합은 먹등 키의 대상이 무엇이어서 하는지를 보여준다. 민간 추천에서 같은 사용자의 클릭이 두 번 기록되면 피처가 약간 편향될 뿐이지만, 공공에서 같은 민원이 대면·전화·온라인 세 채널로 접수되면 그것은 **수송 중복이 아니라 업무 중복**이다. 재전송 중복(수송)과 다채널

중복(업무)은 원인이 다른데 소비자 앞에서는 똑같이 “같은 `complaint_id` 가 다시 온 것”으로 보이고, 5장의 업무 식별자 기반 맥등 키가 둘을 구분 없이 흡수한다(14장 드릴 D2가 이를 실측했다 — 흡수 11건 중 수송 10 + 업무 1). 여기에 공공만의 요구가 하나 더 추가된다: 채널마다 처리 속도·수용률이 다르면 특정 집단(예: 온라인에 익숙하지 않은 고령층이 물리는 대면 채널)이 불리해질 수 있으므로, 채널별 처리 격차를 형평성 지표로 감시해야 한다(16장).

요컨대 전달 보증 수준의 선택은 민간에서 기술·비용 결정이지만, 공공에서는 **검수 문서에 “왜 이 보증 수준인가”가 남고, 유실·중복의 흔적이 실행 산출물로 증거화되며, 그 증거가 감사·형평성 판단의 입력이 되는** 책임성의 결정이다. 방법 자체는 민간과 같으므로 어느 현장에서도 적용되되, 공공에서는 그 위에 증거의 의무가 한 겹 더 추가된다는 것 — 이것이 이 책 전체에서 반복될 적용의 패턴이다.

역방향: 이 장의 설계를 민간 현장에 옮기면

지금까지는 민간 패턴을 공공으로 옮겼다. 반대 방향도 성립한다 — 이 장에서 만든 설계를 그대로 민간 현장에 적용하면 무엇이 바뀌고 무엇이 유지되는지 세 현장에서 확인한다.

이커머스 주문·결제 스트리밍에서는 주문 생성·결제 완료·배송 시작을 각각 토픽으로 나눈다. 명명 규칙은 그대로라 `commerce.order.created.v1` 이 되고, 파티션 키 선택의 구조도 그대로다 — 주문 ID로 지정하면 한 주문의 상태 변화 순서가, 고객 ID로 지정하면 한 고객의 주문 순서가 보존된다. 바뀌는 것은 보증 수준의 근거다. 결제 이벤트에 `at-least-once` + 맥등 반영을 사용하는 이유가 공공에서는 “누락이 곧 행정 처리 누락”이지만 여기서는 “매출과 고객 신뢰”다.

광고 클릭 수집과 과금은 유실·중복의 대가가 양방향으로 금액에 직결되는 경우다. 클릭 1건이 유실되면 광고주에게 과소 청구, 중복되면 과다 청구다. 실습 시나리오 C에서 관찰한 중복 10건이 여기서는 이중 청구 10건이 된다. 대응도 이 책의 순서 그대로다 — `at-least-once`로 유실을 막고, 클릭 ID 기반 UPSERT로 중복을 흡수해(5장) “최소 한 번 전달 + 정확히 한 번 반영”을 만든다. 그리고 공공의 감사 적재 그룹이 놓였던 자리에 정산 배치용 그룹과 ML 학습 적재용 그룹이 들어간다.

실시간 마케팅 트리거(장바구니 이탈 후 30분 내 쿠폰 발송)는 4.5에서 본 커밋 시점 판단이 그대로 재현되는 현장이다. 이 셋을 관통하는 것은 하나다 — 토픽 명명, 파티션 키, 전달 보증, 그룹 분리라는 네 가지 결정은 어느 현장에서도 똑같이 내려야 하고, 각 결정의 **근거만 도메인에 따라 달라진다**. 공공에서 그 근거가 감사·형평성이라면, 민간에서는 매출·비용·SLA다.

한 가지는 방향이 비대칭이다. 민간 설계를 공공에 도입할 때는 증거 산출 의무가 한 겹 추가되지만, 공공 설계를 민간에 도입할 때는 그 의무를 제외해도 손실이 없다 — 감사 적재 그룹은 분석 자산이 되고, 대사 자동화는 정산 정확도가 된다. 공공 요구에 맞춰 설계하면 민간 요구는 대체로 함께 충족된다.

핵심 정리

전달 보증의 선택 원리는 도메인 불변이지만, 유실을 감내하는 정도는 도메인에 따라 달라진다 — 공공은 대체로 더 강한 보증에서 출발한다.

공공에서는 전달 보증 선택이 책임성의 결정이라, “왜 이 수준인가”와 유실·중복의 흔적이 검수·감사 산출물로 증거화되어야 한다.

수송 중복과 업무 중복은 같은 맥등 키로 흡수되며, 채널 통합에는 형평성 감시가 함께 수반된다.

토픽 명명·파티션 키·전달 보증·그룹 분리라는 네 결정은 도메인과 무관하게 내려야 하고, 근거만 도메인에 따라 달라진다.

실습 4.1 민원 이벤트 수집 파이프라인 설계와 실행(★★★☆☆)

이 실습은 **설계 → 실행 → 대조 → 위험 식별** 네 단계로 진행한다. 순서에 의도가 있다. 이미 정해진 토픽으로 코드부터 실행하면 파티션 키가 왜 그 값인지 묻지 않게 되고, 그러면 실행 결과는 “근거 없는 수치”로 남는다. 먼저 자기 판단으로 설계하고 분포를 예측한 다음 실행 로그와 대조하면, 어긋난 지점이 곧 자기 설계의 결함이 된다. 감사·검수는 그 뒤 4단계에서 다룬다 — 설계 근거와 실행 증거가 모두 확보되어야 검수 질문에 답할 수 있기 때문이다.

실습 목표

수집할 이벤트의 토픽 이름·파티션 키·파티션 수·보존 기간을 **스스로 정하고 근거를 적는다** (설계).

실제 브로커에서 Producer/Consumer를 실행해 파티션 배정·순서·지연을 관찰한다(실행).

설계 단계에서 예측한 파티션 분포를 실행 로그와 대조하고, 어긋난 원인을 설명한다(대조).

커밋 시점을 바꾼 두 크래시 시나리오로 유실과 중복을 **실제로 발생시켜** 관찰 보고서를 만든다(증거).

데이터

민원 이벤트는 시뮬레이션 입력이다(개인정보가 포함된 실제 민원 데이터는 사용할 수 없다). 이 실습의 관찰 대상은 데이터 내용이 아니라 **실제 브로커의 전달 동작**이며, 아래 인용하는 수치(파티션 분포, 유실·중복 건수, 지연)는 모두 실제 실행 로그에서 산출된 것이다.

1단계 설계 — 토픽 설계표와 파티션 분포 예측

코드를 실행하기 전에 설계를 먼저 확정한다. 아래 요구사항을 읽고 **표 4.6의 빈칸을 직접 채운다**. 표 4.2는 완성된 예시 답안이므로 그대로 옮겨 적지 않는다 — 채운 뒤에 비교한다.

요구사항(가상 발주 조건) - 서울시 8개 자치구에서 민원이 접수된다. 접수 이벤트에는

`complaint_id`, `district`, `category`, `received_at` 이 담긴다. - 구청별 일일 민원 처리 현황판이 접수 순서대로 목록을 제시한다. - 감사 요구: 접수 누락 여부를 사후에 확인할 수 있어야 한다. 확인 주기는 주 1회다. - 재난 시 특정 구의 민원이 평시의 수집 배로 증가할 수 있다.

표 4.6 토픽 설계표 — 학습자 작성용

항목	내가 정한 값	근거(요구사항의 어느 문장에서 나왔는가)
토픽 이름		
파티션 키		
순서 보장 단위		
파티션 수		
보존 기간		
이 설계가 깨지는 조건		

작성할 때 지킬 것이 세 가지다. 첫째, 근거 칸에 “기본값이라서” 또는 “보통 그렇게 한다”를 쓰지 않는다(4.2 검수 포인트). 둘째, 보존 기간은 디스크 사정이 아니라 감사 주기에서 역산한다. 셋째, “이 설계가 깨지는 조건”에는 재난 버스트처럼 자기 선택이 불리해지는 상황을 적는다 — 트레이드오프를 이해하고 선택했는지 확인하는 칸이다.

파티션 분포 예측: 실행 스크립트는 토픽 `public.complaint.received.v1` 에 파티션 3개, 지역구 키로 200건을 보낸다. 실행하기 **전에** 파티션별 예상 건수를 적어 둔다.

	파티션 0	파티션 1	파티션 2
내 예측			
실제(3단계에서 기입)			

예측 근거도 한 줄 적는다. 8개 구가 3개 파티션에 어떻게 나뉘는지, 그리고 그 결과 건수가 고르게 분포하는지 여부를 판단하면 된다.

2단계 실행 — 브로커에서 3개 시나리오 실행

3장 최소 구성의 브로커를 재사용한다.

```
cd practice/chapter3 && docker compose up -d zookeeper kafka
cd ../chapter4
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter4.py
```

오케스트레이터(`run_chapter4.py`)는 세 시나리오를 차례로 실행하고 관찰 보고서를 생성한다.

- A. baseline:** 컨슈머를 먼저 실행한 뒤 200건 생산 — 정상 경로의 기준선.
- B. at-most-once + 크래시:** 커밋을 처리보다 먼저 하는 컨슈머가 10건 처리 후 강제 종료(exit 42), 재시작해도 커밋 위치 뒤라 남은 이벤트를 받지 못함(유실).
- C. at-least-once + 크래시:** 처리를 커밋보다 먼저 하는 컨슈머가 10건 처리 후 강제 종료, 재시작 후 옛 커밋 위치부터 다시 읽어 이미 처리한 건을 재수신(중복).

크래시는 `os._exit()` 로 재현한다 — 커밋도 그룹 탈퇴 통지도 없이 중단되는, 실제 장애와 같은 조건이다.

핵심 코드

커밋 시점 차이의 구현은 다음이 전부다.

```
if mode == "at-most-once":
    consumer.commit()          # 처리 "전" 커밋 → 크래시 시 유실
    process(batch)
if mode == "at-least-once":
    consumer.commit()          # 처리 "후" 커밋 → 크래시 시 중복
```

전체 코드는 `practice/chapter4/code/4-2-kafka-consumer.py`, 오케스트레이션은 `practice/chapter4/run_chapter4.py` 참고

산출물 안내

산출물	내용	용도
<code>ch4_delivery_report.json</code>	시나리오별 생산·처리·유실·중복·지연	메시지 유실·중복·지연 관찰 보고서 — 검수 자산
<code>ch4_produce_*.json</code>	전송 요약(파티션 분포, 키→파티션 매핑)	4.3 실행 결과의 원본 증거
<code>ch4_consume_*.json</code> / <code>ch4_processed_*.jsonl</code>	소비 요약 / 이벤트별 처리 기록	4.4 실행 결과·중복 판별의 원본 (5장 실습의 입력이 된다)
표 4.6 토픽 설계표(학습자 작성)	토픽 이름·키·파티션 수·보존과 각각의 근거	토픽 설계표 — 검수 자산, 3단계 대조 후 확정

실행 중 확인 체크리스트

- ☐ A: 유실 0·중복 0이고, 같은 지역구가 항상 같은 파티션인가
- ☐ B: 컨슈머가 exit 42로 중단된 뒤 재시작 컨슈머가 **아무것도 받지 못했는가**(유실 > 0)
- ☐ C: 재시작 컨슈머가 처음부터 다시 읽어 **중복이 기록되었는가**(중복 > 0, 유실 0)
- ☐ B·C 모두: 크래시 후 재시작 사이에 12초 대기가 있는 이유를 말할 수 있는가(세션 타임아웃)
- ☐ 실행 전에 1단계 설계표와 파티션 분포 예측을 적어 두었는가(실행 후에 적으면 대조가 성립하지 않는다)

2단계 실행 결과(실제)

관찰 보고서는 `practice/chapter4/data/output/ch4_delivery_report.json` 에 저장된다.

표 4.7 시나리오별 유실·중복·지연 관찰 결과(실제 실행)

시나리오	생산	처리 기록	고유 이벤트	중복	유실	비고
A. baseline	200	200	200	0	0	지연 평균 3.5ms·최대 110.7ms
B. at-most-once+크래시	30	10	10	0	20	커밋 먼저 → 유실
C. at-least-once+크래시	30	40	30	10	0	처리 먼저 → 재수신 중복

B에서는 확보한 30건이 모두 커밋된 상태에서 10건만 처리하고 중단되었다. 재시작 컨슈머는 커밋 위치(30) 이후부터 읽으므로 아무것도 받지 못했다 — 20건이 조용히 유실되었다. (실습 스크립트는 크래시 지점이 폴링 배치 경계와 우연히 일치해 유실이 0이 되는 것을 막도록, 크래시 전에 처리량보다 많은 레코드를 확보·커밋해 둔다.)

C에서는 커밋 없이 10건을 처리하고 중단되었다. 재시작 컨슈머는 처음부터 30건을 다시 읽어, 이미 처리된 10건이 중복 기록됐다(처리 기록 40 = 10 + 30). 고유 이벤트는 30건 — 유실은 없다.

유실·중복 건수는 폴링 배치 크기와 크래시 시점에 따라 실행마다 달라진다. 학습 목표는 특정 숫자가 아니라 **방향**이다: B는 반드시 유실 쪽으로, C는 반드시 중복 쪽으로 실패한다.

3단계 대조 — 설계 예측과 실행 결과의 대조

1단계에서 적어 둔 예측표의 “실제” 줄을 `ch4_produce_baseline.json` 의 `partition_distribution` 값으로 채운다. baseline 200건의 실측값은 다음과 같다.

	파티션 0	파티션 1	파티션 2
균등 분산이라면	66.7	66.7	66.7
실제	61	76	63

파티션 1이 다른 둘보다 약 20% 많다. 원인은 부하가 아니라 **키의 개수와 파티션 수가**

나누어떨어지지 않는다는 것이다. 같은 파일의 `district_to_partitions` 를 보면 8개 구가 3개 파티션에 2:3:3으로 분배되었다 — 관악·성동이 파티션 0, 강남·마포·중이 파티션 1, 송파·용산·종로가 파티션 2다. 구마다 이벤트 수가 비슷하면 3개 구를 받은 파티션이 2개 구를 받은 파티션보다 1.5배를 받는 셈이고, 실측 61:76:63이 그 비율에 가깝다.

여기서 확인할 것 세 가지다.

예측이 균등(약 67씩)이었다면 무엇을 빠뜨렸는가. 키 기반 파티셔닝은 이벤트를 고르게 분산시키는 장치라 아니라 같은 키를 같은 파티션에 묶는 장치다. 부하 균등은 키 개수가 파티션 수보다 충분히 많고 키별 빈도가 비슷할 때 발생하는 부수 효과일 뿐이다.

키 → 파티션 매핑이 실행 내내 유지되었는가. 같은 파일의 `key_to_single_partition` 이 `true` 면 각 구의 이벤트가 한 파티션에만 들어갔다는 뜻이고, 곧 구별 접수 순서가 보존되었다는 뜻이다. 1단계에서 “순서 보장 단위 = 지역구별 접수 순서”라고 적었다면 이

필드가 그 설계의 실행 증거다.

재난 버스트를 적용하면 이 표가 어떻게 바뀌는가. 강남구 하나의 이벤트가 10배로 증가하면 파티션 1의 부하만 증가한다. 나머지 둘에 여유가 남아도 그 부하를 분담할 방법이 없다 — 같은 키는 같은 파티션이라는 성질이 여기서는 비용으로 작용한다.

1단계 표 4.6로 돌아가 “이 설계가 깨지는 조건” 칸을 다시 쓴다. 처음에 비워 두었거나 막연히 적었다면, 지금은 실측 분포를 근거로 작성할 수 있다. **설계표의 최종본은 이 수정을 거친 것이며, 과제로 제출하는 것도 이 최종본이다.**

관찰

크래시 직후 재시작한 컨슈머는 곧바로 파티션을 받지 못했다. 중단된 이전 컨슈머가 세션 타임아웃(10초)으로 그룹에서 제외될 때까지 재할당이 지연되기 때문이다(실습 스크립트는 12초를 기다린 뒤 재시작한다). “장애 복구 시간”에는 프로세스 재기동만이 아니라 그룹 재할당 대기가 포함된다.

C의 중복 10건은 버그가 아니라 at-least-once의 **정의대로의 동작**이다. 이 중복을 어떻게 제거할 것인가 — event_id 같은 중복 제거 키, 저장소의 유니크 제약 — 가 5장의 주제다.

4단계 위험 식별 — 설계와 증거를 검수 언어로 옮기기

앞의 세 단계에서 설계 근거(1단계)와 실행 증거(2·3단계)를 모두 확보했다. 검수·감사가 요구하는 것은 그 둘을 이어 붙인 형태다 — “무엇이 잘못될 수 있는가, 그렇게 판단한 근거가 어느 산출물의 어느 값인가, 무엇으로 막는가”. 아래 표의 세 행은 실행 증거에서, 넷째 행은 1단계 설계에서 나온다.

위험	실습에서의 근거	대응 방향
커밋 시점 오설계로 인한 조용한 유실	시나리오 B: 유실 20건, 오류 로그 없음	at-least-once 기본값, 커밋은 처리 후
중복 처리로 인한 이중 집계	시나리오 C: 중복 10건	중복 제거 키·Upsert(5장)
재시작 후 소비 정지 오인	세션 타임아웃 전 재할당 지연	복구 절차에 재할당 대기 반영, 랙 모니터링(11장)
재난 버스트 시 핫 파티션	3단계 대조: 8개 키가 2:3:3으로 갈려 61:76:63, 한 구가 폭증하면 한 파티션만 커짐	순서 보장 단위 재검토(민원 건 ID), 파티션 수 여유, 파티션별 랙 감시

셋째 열이 비어 있거나 “모니터링한다” 같은 말로 채워지면 그 행은 검수 문서에서 유용하지 않다. 대응 방향에는 **바꿀 설정값이나 바꿀 설계 결정**이 들어가야 한다.

제출물 3종: ① 표 4.6 토픽 설계표(3단계 수정 반영한 최종본), ② ch4_delivery_report.json, ③ 위 위험 식별 표. ①이 설계, ②가 증거, ③이 둘을 이은 검수 자산이다.

보충(전문가 트랙 포인터)

멥등 Producer와 순서 보장 조건: `enable.idempotence=true` 는 Producer 재시도 중복을 브로커가 제거하게 한다(최신 Kafka는 KIP-679로 이를 기본값으로 켜다). 순서 보장과 동시 전송 수(`max.in.flight.requests`)의 조건이 함께 붙는다. `exactly-once`로의 확장(트랜잭션)은 KIP-98의 설계다.

Schema Registry와 스키마 진화: 토픽 버전 전환(4.2) 대신 스키마 호환성 규칙을 중앙에서 관리하는 방식.

운영 보안: 실습 브로커는 평문(PLAINTEXT) 통신이다. 운영에서는 TLS 암호화, SASL 인증, 토픽 단위 ACL이 기본이다(3장 3.3의 보안 요구를 Kafka 설정으로 전환하는 문제).

핵심 용어

토픽/파티션/오프셋: 이벤트 채널 / 채널을 나눈 순서 로그 / 로그 내 순번

파티션 키: 파티션 배정 기준이자 순서 보장 단위

`acks(0/1/all)`: “전송 성공”의 정의 — 확인 없이 / 리더 기록 / 동기화 복제본 전체

멥등 Producer: `enable.idempotence` 로 세션 내 재시도 중복을 브로커가 제거(최신 기본값 — KIP-679)

컨슈머 그룹/커밋: 파티션을 나눠 맡는 소비자 집합 / “어디까지 읽었는지”의 기록

컨슈머 랙(lag): 마지막 오프셋 – 커밋 오프셋 — 처리 지연의 조기 신호(SLI)

`at-most-once` / `at-least-once` / `exactly-once`: 커밋 먼저(유실 가능) / 처리 먼저(중복 가능) / 정확히 한 번 반영(멥등·트랜잭션)

세션 타임아웃/리밸런싱: 중단된 소비자 감지 창 / 파티션 재할당

리플레이: 오프셋을 되돌려 보존 기간 내 이벤트를 재처리 — 감사 재현의 토대

대사(reconciliation): 생산 건수와 처리 건수를 대조하는 절차 — 로그가 남지 않는 유실을 검출하는 유일한 수단

다음 장 예고

이 장의 시나리오 C는 `at-least-once`가 만들어 내는 중복을 실측으로 남겼다. 5장은 이 중복이 어디서(Producer·브로커·Consumer·DB) 왜 발생하는지 단계별로 분석하고, 중복 제거 키 설계와 저장소 Upsert·유니크 제약으로 “최소 한 번 전달 + 정확히 한 번 반영”을 만드는 방법을 다룬다. 4장이 남긴 실측 중복 스트림이 그 5장 실습의 입력이 된다.

참고문헌

기존(직접 선례 — Kafka 공식·클라이언트)

Apache Kafka. (n.d.). *Introduction*. <https://kafka.apache.org/intro>

Apache Kafka. (n.d.). *Documentation: Message Delivery Semantics*.
<https://kafka.apache.org/documentation/#semantics>
Apache Kafka. (n.d.). *Documentation: Producer Configs*.
<https://kafka.apache.org/documentation/#producerconfigs>
Apache Kafka. (n.d.). *Documentation: Consumer Configs*.
<https://kafka.apache.org/documentation/#consumerconfigs>
Confluent. (n.d.). *Message Delivery Guarantees*.
<https://docs.confluent.io/kafka/design/delivery-semantics.html>
dppk/kafka-python. (n.d.). *GitHub repository*. <https://github.com/dppk/kafka-python>

보강(전달 보증·역등·exactly-once — 2026-07-11 URL 실검증)

Apache Kafka. (n.d.). *Documentation: Exactly Once Semantics*.
<https://kafka.apache.org/documentation/#exactlyonce> — 직접 선례(역등 Producer·트랜잭션의 공식 설명).
Apache Kafka Project. (2017). *KIP-98 — Exactly Once Delivery and Transactional Messaging*.
<https://cwiki.apache.org/confluence/display/KAFKA/KIP-98+-+Exactly+Once+Delivery+and+Transactional+Messaging> — 직접 선례(트랜잭션 설계 문서).
Apache Kafka Project. (2021). *KIP-679 — Producer will enable the strongest delivery guarantee by default*. <https://cwiki.apache.org/confluence/display/KAFKA/KIP-679%3A+Producer+will+enable+the+strongest+delivery+guarantee+by+default> — 직접 선례(최신 Producer 기본값: `enable.idempotence=true · acks=all`).
Narkhede, N. (2017). *Exactly-once Semantics are Possible: Here's How Kafka Does It*.
Confluent Blog. <https://www.confluent.io/blog/exactly-once-semantics-are-possible-heres-how-apache-kafka-does-it/> — 배경·구현 해설(exactly-once 반영의 원리).
Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.
<https://dataintensive.net/> — 배경 자료(분산 시스템 일반 원칙: 로그 복제, ack 모호성, 전달 보증). 직접 선례가 아니라 개념 배경으로 인용.